

栈、队列与受限访问

减少端点自由度，怎样生成可推理的时间秩序

408 · 数据结构 Topic DS-02

母问题：为什么“少允许一些操作”反而能表达过程？

一般线性表允许在许多位置访问和修改；栈、队列与双端队列主动收紧这个自由度。它们的生成链是

Access Restriction → Temporal Order → State Transition

→ Invariant → Application Contract.

这里的箭头依次表示因果生成和检查顺序：先规定哪些端点可进、可出，才能推出元素被取出的时间秩序；每次操作再更新状态字段，并保持空、满、首尾指针和元素次序不变量。应用只依赖这份契约，不应依赖底层是数组还是链表。

本册定位与 Owner 边界

- **Owns**：栈、队列、循环队列、链队列、双端队列的 ADT 契约、物理状态、操作生命周期、空满判据与复杂度。
- **Uses**：DS01 的顺序/链接表示；Atlas 的对象—操作—不变量—成本语言。
- **Outputs**：向 DS-B01 输出可替换的 Frontier 容器契约；向 DS04/DS07 提供遍历所需的取出顺序。
- **Stops**：递归语义、表达式文法、树/图遍历正确性和单调结构的淘汰规则不由本册拥有；这里只说明它们为何能够调用受限访问容器。

目录

1	从线性表到受限访问 ADT	3
2	栈：顶端是唯一活动边界	3
3	队列：两端分工生成 FIFO	3
3.1	为什么顺序队列需要循环化	4
3.2	空与满必须增加一份信息	4
4	双端队列：两端自由，但状态仍受约束	4
5	代码把访问契约落实为状态机	4
5.1	顺序栈：一份 size 同时分隔有效区与空闲区	4
5.2	循环队列：每次操作只改变一个端点	5
5.3	链队列：删除唯一节点时修复两个端点	6
5.4	双端队列：前端与后端的顺序不对称	7
5.5	边界测试：逼状态跨过零、满与数组末端	9

6	复杂度必须绑定表示与输入契约	10
7	应用只调用时间秩序，不接管容器机制	10
8	概念边界：相似端点名不能替代定义	10
9	做题控制协议与压缩复原	11
10	全科坐标与来源处理	11

1 从线性表到受限访问 ADT

若线性表的状态是一个序列 $S = (a_1, \dots, a_n)$ ，那么“栈”和“队列”的区别不在元素类型，而在允许观察和删除哪一端：

ADT	允许进入	允许离开/观察	推出的次序	典型过程含义
栈 Stack	顶端	顶端	LIFO，后进先出	最近未完成的上下文
队列 Queue	队尾	队头	FIFO，先进先出	最早等待的任务
双端队列 Deque	两端	两端	由调用者选择	可同时模拟栈和队列

限制不是性能结论。栈的 push/pop 可以由顺序表尾部实现，也可以由链表头部实现；只要返回序列一致，二者兑现的是同一 ADT。表示改变的是容量、分配、局部性和维护成本，不是 LIFO/FIFO 的抽象结果。

完整生成例：三项任务按栈和队列退出

依次加入 A, B, C 。若只允许从加入端退出，则状态轨迹为

$$[] \rightarrow [A] \rightarrow [A, B] \rightarrow [A, B, C] \rightarrow [A, B] \rightarrow [A] \rightarrow [],$$

退出序列是 C, B, A 。若一端加入、另一端退出，则退出序列是 A, B, C 。同一组元素没有改变，访问限制却确定了时间秩序；这正是应用选择容器时真正购买的语义。

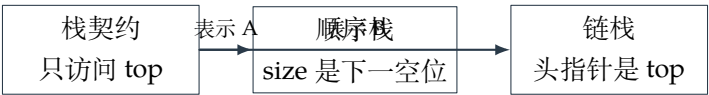
2 栈：顶端是唯一活动边界

把栈状态写作 $(data, size, capacity)$ 或 $(top, size)$ 。顺序栈的合法状态是

$$0 \leq size \leq capacity, \quad top = data[size - 1] \quad (size > 0).$$

执行 push(x) 时先判满，再写入 $data[size]$ 并递增长度；执行 pop() 时先判空，再递减长度并读取原顶元素。**先减后读与先读后减**都可正确实现，但下标和返回值必须指向同一个旧顶元素。

链栈把头节点作为栈顶：压栈令新节点的后继指向旧顶，再更新顶指针；出栈先保存旧顶，再跨过它并释放。若把尾节点当栈顶而底层又是单链表，出栈还要寻找前驱，便丢掉了预期的 $O(1)$ 局部更新。



3 队列：两端分工生成 FIFO

队列允许在队尾 enqueue、在队头 dequeue。链队列通常保存 $(front, rear, size)$ ：入队接到 $rear$ 后，出队从 $front$ 删除。最危险的不是一般节点，而是最后一个节点出队：新 $front$ 为空时， $rear$ 也必须复位为空，否则“空队列”会残留悬空尾指针。

3.1 为什么顺序队列需要循环化

若数组队列只让 $front, rear$ 单调右移，前部空槽无法复用，会出现“数组还有空间但队尾到界”的假溢出。循环队列把下标放在模 M 的环上：

$$\text{next}(i) = (i + 1) \bmod M.$$

这不是为了背取模公式，而是把已出队的槽重新纳入可用状态空间。

3.2 空与满必须增加一份信息

只保存 $front, rear$ 且二者都指向下一操作位置时， $front = rear$ 既可能表示空，也可能表示绕一圈后的满。实现必须选择一种消歧协议：

协议	空判据	满判据	代价/适用点
牺牲一个槽	$front = rear$	$\text{next}(rear) = front$	可用容量为 $M - 1$
长度计数	$size = 0$	$size = M$	多维护一个计数字段
标志/最近操作	$front = rear$ 且标志为空	$front = rear$ 且标志为满	分支语义更复杂

本册伴随实现选择长度计数，因此合法状态还要求

$$0 \leq size \leq M, \quad rear \equiv front + size \pmod{M}.$$

不能把该实现的 $size = M$ 满判据与“牺牲一个槽”实现的 $\text{next}(rear) = front$ 混用。

4 双端队列：两端自由，但状态仍受约束

双端队列允许 $\text{push_front}/\text{push_back}$ 与 $\text{pop_front}/\text{pop_back}$ 。栈和普通队列都可由它限制操作集合得到：

$$\text{Deque} \xrightarrow{\text{只在同端进出}} \text{Stack}, \quad \text{Deque} \xrightarrow{\text{一端进、另一端出}} \text{Queue}.$$

这说明 Deque 的 ADT 能力更强，却不等于所有应用都应使用它。更宽的接口允许更多非法调用组合；若应用只需要 FIFO，直接暴露 Queue 契约更能保护时间秩序。

循环双端队列仍维护 $rear \equiv front + size \pmod{M}$ 。前端插入要先令 $front$ 向前回绕再写入；后端删除要先令 $rear$ 向前回绕再读取。顺序写反会读到“下一空位”，产生典型的 off-by-one 错误。

5 代码把访问契约落实为状态机

完整 C++17 实现位于 `code/ds02_restricted_access.hpp`，测试位于 `code/ds02_restricted_access_test.cpp`。以下代码块直接抽取真实源文件，正文解释与可执行实现共享同一来源。

5.1 顺序栈：一份 size 同时分隔有效区与空闲区

Listing 1: 顺序栈的空满、压栈、出栈与不变量

17

```
bool empty() const { return size_ == 0; }
```

```

18  bool full() const { return size_ == capacity_; }
19  std::size_t size() const { return size_; }
20
21  void push(int value) {
22      if (full()) {
23          throw std::overflow_error("stack overflow");
24      }
25      data_[size_++] = value;
26  }
27
28  int pop() {
29      if (empty()) {
30          throw std::underflow_error("stack underflow");
31      }
32      return data_[--size_];
33  }
34
35  int top() const {
36      if (empty()) {
37          throw std::underflow_error("stack is empty");
38      }
39      return data_[size_ - 1];
40  }
41
42  bool invariant() const {
43      return size_ <= capacity_ && (capacity_ == 0 || data_ != nullptr);
44  }

```

代码把有效区定义为 $[0, size)$ ，因此压栈写 `data_[size_]` 后递增，出栈递减后读取。满/空检查在访问数组之前执行，容量为 0 时也只会进入上溢分支，不会解引用空存储。

5.2 循环队列：每次操作只改变一个端点

Listing 2: 循环队列的入队、出队与环形不变量

```

128  bool empty() const { return size_ == 0; }
129  bool full() const { return size_ == capacity_; }
130  std::size_t size() const { return size_; }
131
132  void enqueue(int value) {
133      if (full()) {
134          throw std::overflow_error("queue overflow");
135      }
136      data_[rear_] = value;
137      rear_ = next(rear_);
138      ++size_;
139  }
140

```

```

141 int dequeue() {
142     if (empty()) {
143         throw std::underflow_error("queue underflow");
144     }
145     const int value = data_[front_];
146     front_ = next(front_);
147     --size_;
148     return value;
149 }
150
151 int front() const {
152     if (empty()) {
153         throw std::underflow_error("queue is empty");
154     }
155     return data_[front_];
156 }
157
158 bool invariant() const {
159     if (size_ > capacity_) {
160         return false;
161     }
162     if (capacity_ == 0) {
163         return data_ == nullptr && front_ == 0 && rear_ == 0 && size_ == 0;
164     }
165     return data_ != nullptr && front_ < capacity_ && rear_ < capacity_ &&
166            rear_ == (front_ + size_) % capacity_;
167 }

```

入队只写 *rear* 所指的下一空位并推进 *rear*；出队只读 *front* 所指的首元素并推进 *front*。两者分别增减 *size*，共同维护 $rear = (front + size) \bmod capacity$ 。next 对零容量单独返回 0，但正常操作在此前已由空满条件拦截。

5.3 链队列：删除唯一节点时修复两个端点

Listing 3: 链队列的首尾迁移与不变量

```

196 void enqueue(int value) {
197     Node* node = new Node{value, nullptr};
198     if (rear_ == nullptr) {
199         front_ = rear_ = node;
200     } else {
201         rear_->next = node;
202         rear_ = node;
203     }
204     ++size_;
205 }
206
207 int dequeue() {

```

```

208     if (empty()) {
209         throw std::underflow_error("queue underflow");
210     }
211     Node* removed = front_;
212     const int value = removed->value;
213     front_ = removed->next;
214     if (front_ == nullptr) {
215         rear_ = nullptr;
216     }
217     delete removed;
218     --size_;
219     return value;
220 }
221
222 int front() const {
223     if (empty()) {
224         throw std::underflow_error("queue is empty");
225     }
226     return front_->value;
227 }
228
229 bool invariant() const {
230     if ((size_ == 0) != (front_ == nullptr && rear_ == nullptr)) {
231         return false;
232     }
233     std::size_t count = 0;
234     Node* last = nullptr;
235     for (Node* current = front_; current != nullptr; current = current->
next) {
236         last = current;
237         ++count;
238     }
239     return count == size_ && last == rear_ &&
240         (rear_ == nullptr || rear_->next == nullptr);

```

一般出队只移动 front_；当旧队列只有一个节点时，移动后 front_ 为空，代码同步把 rear_ 置空。invariant() 再检查空状态等价、可达节点数、尾节点身份和尾后继为空。

5.4 双端队列：前端与后端的顺序不对称

Listing 4: 循环双端队列的四种端点更新

```

269 void push_front(int value) {
270     require_space();
271     front_ = previous(front_);
272     data_[front_] = value;
273     ++size_;
274 }

```

```

275
276 void push_back(int value) {
277     require_space();
278     data_[rear_] = value;
279     rear_ = next(rear_);
280     ++size_;
281 }
282
283 int pop_front() {
284     require_element();
285     const int value = data_[front_];
286     front_ = next(front_);
287     --size_;
288     return value;
289 }
290
291 int pop_back() {
292     require_element();
293     rear_ = previous(rear_);
294     const int value = data_[rear_];
295     --size_;
296     return value;
297 }
298
299 int front() const {
300     require_element();
301     return data_[front_];
302 }
303
304 int back() const {
305     require_element();
306     return data_[previous(rear_)];
307 }
308
309 bool invariant() const {
310     if (size_ > capacity_) {
311         return false;
312     }
313     if (capacity_ == 0) {
314         return data_ == nullptr && size_ == 0 && front_ == 0 && rear_ == 0;
315     }
316     return data_ != nullptr && front_ < capacity_ && rear_ < capacity_ &&
317         rear_ == (front_ + size_) % capacity_;

```

front_ 始终指向首元素，rear_ 始终指向尾后下一空位。因此前插是“先退再写”，后插是“先写再进”；前删是“先读再进”，后删是“先退再读”。四个操作不是可随意对称替换的口诀，而是由端点语义生成。

5.5 边界测试：逼状态跨过零、满与数组末端

Listing 5: 循环队列满状态、回绕与 FIFO 断言

```

48
49 void test_circular_queue_wraparound() {
50     CircularQueue zero_capacity(0);
51     assert(zero_capacity.empty() && zero_capacity.full());
52     assert_throws<std::overflow_error>([&] { zero_capacity.enqueue(1); });
53     assert(zero_capacity.invariant());
54
55     CircularQueue queue(3);
56     queue.enqueue(10);
57     queue.enqueue(20);
58     queue.enqueue(30);
59     assert_throws<std::overflow_error>([&] { queue.enqueue(40); });
60     assert(queue.dequeue() == 10);
61     queue.enqueue(40);
62     assert(queue.front() == 20);
63     assert(queue.dequeue() == 20);
64     assert(queue.dequeue() == 30);
65     assert(queue.dequeue() == 40);
66     assert(queue.empty() && queue.invariant());
67 }
68
69 void test_linked_queue_singleton() {
70     LinkedQueue queue;
71     assert_throws<std::underflow_error>([&] { queue.dequeue(); });

```

测试先填满容量为 3 的队列，验证第四次入队上溢；再出队一个元素并入队 40，迫使 *rear* 回绕，最后检查退出仍为 10,20,30,40 的 FIFO 次序。测试集还覆盖零容量、空栈/空队列下溢、链队列删至空后复用，以及双端队列两端交错操作。

```

clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
    code/ds02_restricted_access_test.cpp -o /tmp/ds02_test
/tmp/ds02_test

```

6 复杂度必须绑定表示与输入契约

操作	顺序表示	链式表示	成本来源	边界条件
栈 push/pop	$O(1)$	$O(1)$	尾下标或头指针局部更新	定容上溢/动态分配
队列 en-queue/dequeue	$O(1)$	$O(1)$	环形端点或首尾指针	正确空满判据
访问 top/front/back	$O(1)$	$O(1)$	直接端点	非空
检查结构不变量	$O(1)$	$O(n)$	字段关系 vs 全链遍历	属于测试成本
存储 n 个元素	$O(M)$	$O(n)$	定容槽位 vs 每节点指针	$n \leq M$

若顺序栈采用动态扩容，单次扩容为 $\Theta(n)$ ，连续压栈可讨论摊还 $O(1)$ ；本册实现采用固定容量，因而满时显式抛出上溢，不能悄悄套用动态数组结论。

7 应用只调用时间秩序，不接管容器机制

- **递归/括号/表达式**：需要恢复最近尚未完成的上下文，因此调用 LIFO；文法和递归语义由相应专题解释。
- **层序遍历/BFS**：需要先展开较早发现的 frontier，因此调用 FIFO；“首次发现得到无权最短边数”由 DS07 的图模型证明。
- **DFS**：显式栈或调用栈都提供 LIFO frontier；访问标记、完成时序和图性质仍由 DS07 Own。
- **单调栈/单调队列**：它们在端点限制之外又增加“发现更优候选时淘汰旧候选”的不变量；该淘汰规则不是普通栈/队列定义。

Frontier Bridge 的接口语言

调用者只需回答：新发现状态怎样加入？下一状态按什么策略取出？何时算已处理？普通 Stack/Queue 提供前两项的 LIFO/FIFO 策略；DS-B01 负责把这份接口桥接到树和图，DS04/DS07 再提供各自的正确性不变量。

8 概念边界：相似端点名不能替代定义

概念 A	≠ 概念 B	真正判据	混淆后果
ADT 次序	≠ 物理表示	返回序列 vs 数组/节点布局	把实现细节当定义
rear 元素	≠ rear 下标	本册 rear 指向尾后空位	读取错误槽位
队空	≠ front=rear	该等式必须结合消歧协议	把满队误判为空
上溢	≠ 假溢出	真无槽位 vs 未复用前部槽位	错判容量状态
Deque 能力	≠ Queue 契约	能两端操作 vs 只允许 FIFO	应用可破坏时间秩序
容器不变量	≠ 遍历正确性	结构合法 vs 算法输出正确	Owner 越界

最常见的错误推理

“循环队列就是下标取模”漏掉空满消歧；“front/rear 的公式可以直接套”漏掉二者究竟指向元素还是空位；“链式队列不会满”忽略分配失败和资源约束；“BFS 用了队列，所以最短路由队列证明”混淆容器顺序与图算法不变量。

9 做题控制协议与压缩复原

遇到受限访问结构，按以下顺序建模：

1. 写操作集合：哪一端允许进入、离开和观察；
2. 写清端点字段语义，尤其 front/rear 指元素还是空位；
3. 选择空满消歧协议，不混用其他协议的公式；
4. 画一次普通状态和一次跨边界状态，逐步更新字段；
5. 验证抽象退出次序和物理不变量；
6. 最后把复杂度绑定到表示、容量策略和输入前提。

一页压缩

忘掉公式后，从五问复原：允许动哪一端？这生成什么时间秩序？端点字段各指什么？空与满靠哪份额外信息区分？一次操作后哪些字段必须一起修复？

栈用同端进出生成 LIFO；队列用异端进出生成 FIFO；双端队列提供更宽能力。循环化只解决槽位复用，空满消歧才决定状态是否可判；应用正确性必须继续交给调用该容器的 Topic。

10 全科坐标与来源处理**Map Coordinate: Restriction / Order / State / Application**

DS02 接收 DS01 的顺序与链接表示，把“访问自由度”压缩为 LIFO/FIFO/Deque 契约；再经 DS-B01 把取出策略交给 DS04 的树遍历和 DS07 的图遍历。数据结构 Atlas 中“表示—操作—不变量—成本”在本册具体化为端点语义、状态字段与边界分支。

本册吸收归档总册中“减少自由度换取过程语义”“循环队列关键在空满规则而非取模口诀”以及 Frontier 统一视角。旧材料中 BFS/DFS、括号匹配、表达式和单调结构仅作为 Use 证据：其专属机制不提升为 DS02 的稳定定义。伴随实现承担代码级机制证据；测试承担边界与不变量证据。